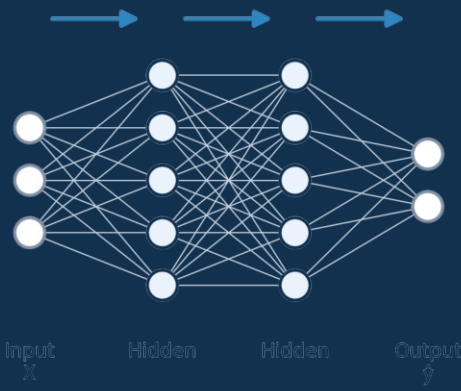
A VISUAL & MATHEMATICAL GUIDE

Neural Networks

From a single neuron to a trained model — layers, forward pass, backpropagation, activation functions, loss, optimizers, and regularization.

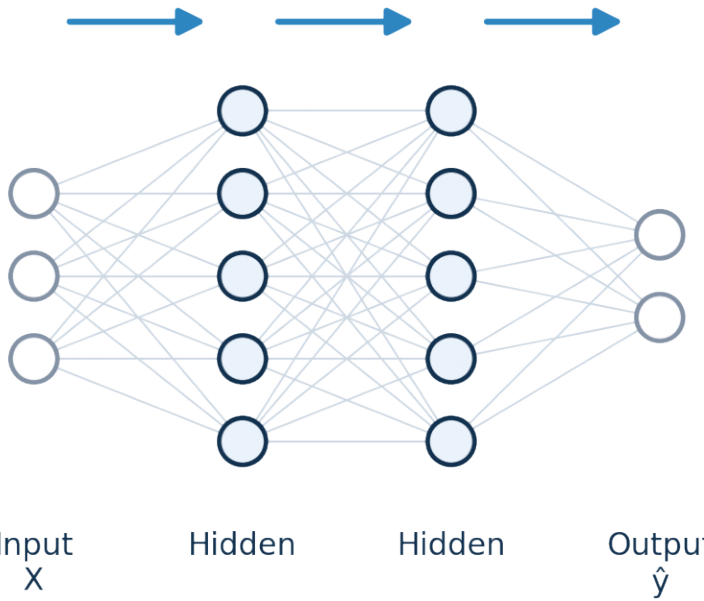


20 slides · the math and the pictures behind how networks learn

What Is a Neural Network?

A neural network is a trainable function that maps input data to a prediction.

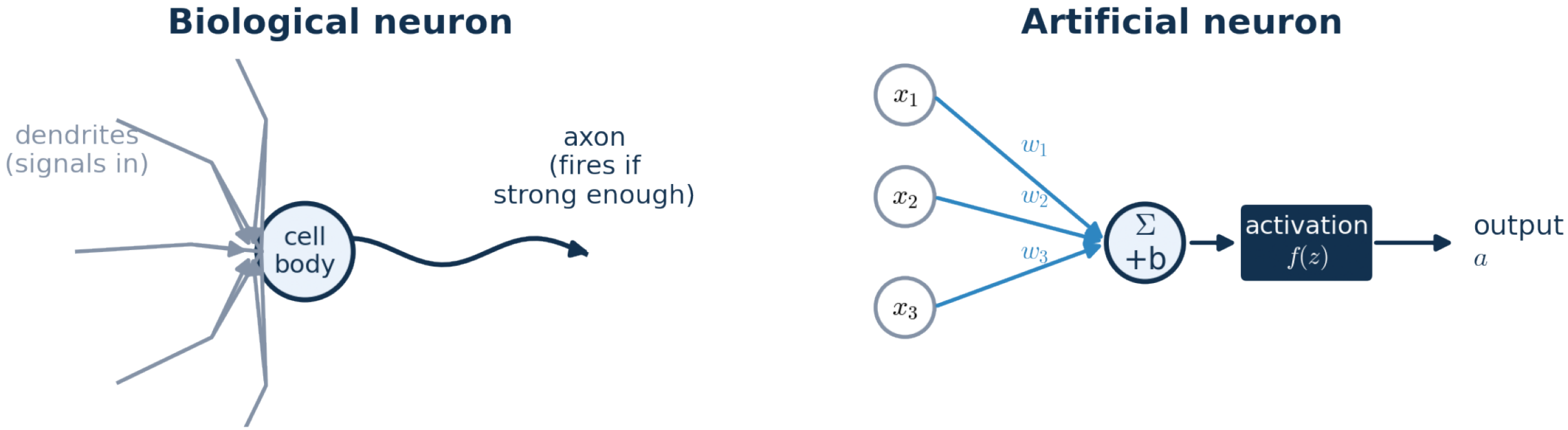
$$\hat{y} = f_\theta(X)$$



KEY IDEA a neural network is a flexible function f with adjustable weights and biases (theta); training tunes theta so predictions match reality.

Biological Neuron → Artificial Neuron

Artificial neurons are a simple mathematical cartoon of a brain cell.

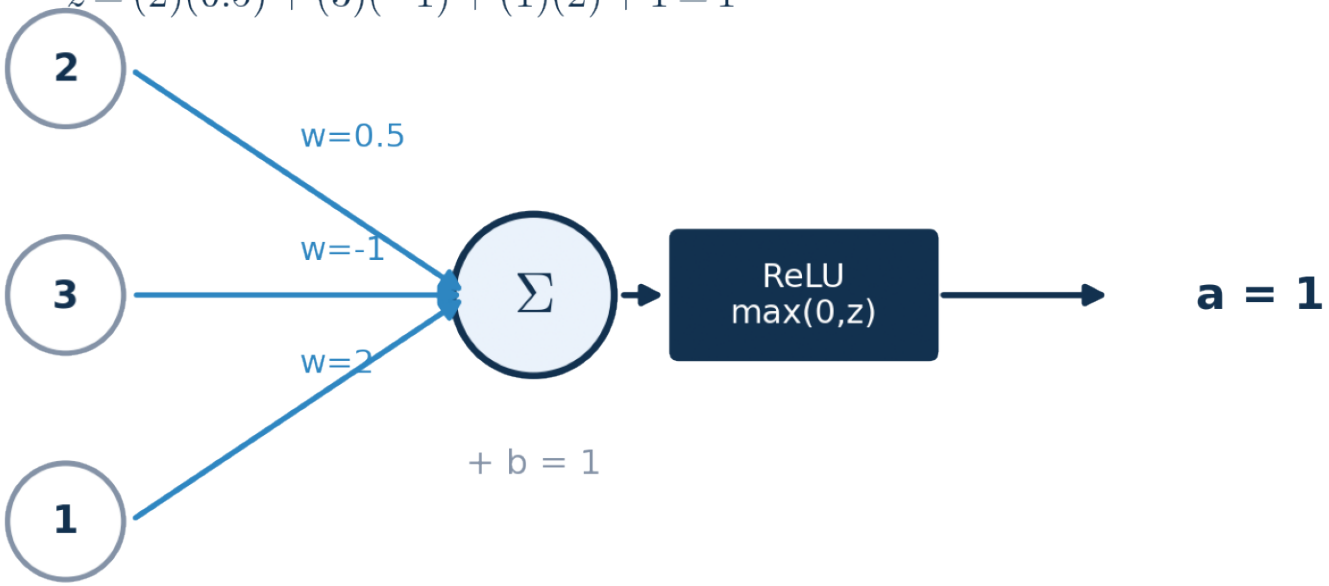


KEY IDEA collect signals, weight them, sum them, add a bias, and decide whether (and how strongly) to fire — that decision is the activation function.

The Single Neuron — Worked Example

A neuron multiplies inputs by weights, adds a bias, then applies an activation.

$$z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$
$$z = (2)(0.5) + (3)(-1) + (1)(2) + 1 = 1$$

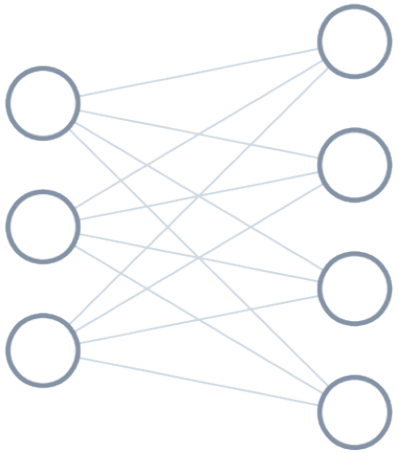


KEY IDEA z = weighted sum + bias is pure linear algebra; the activation (here ReLU) is the only nonlinear step in the whole neuron.

Dense Layer — Matrix Form

A dense layer runs many neurons in parallel using one matrix multiplication.

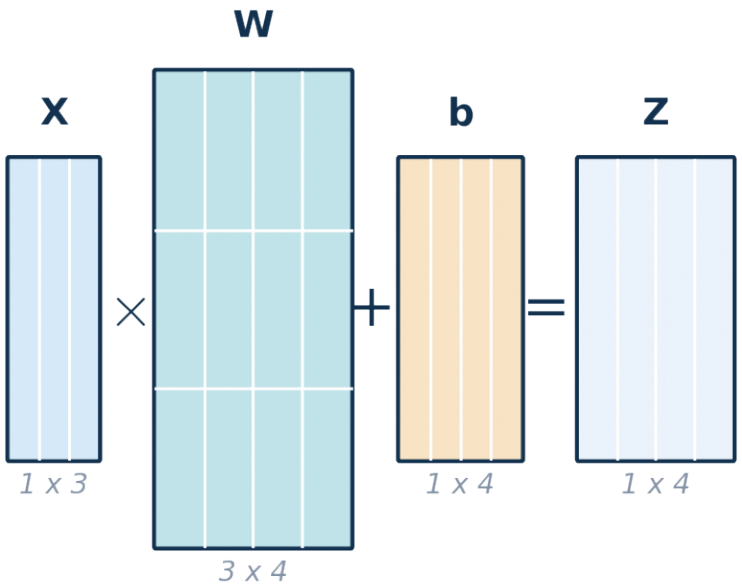
Every input connects to every neuron



Input features (3)

Dense neurons (4)

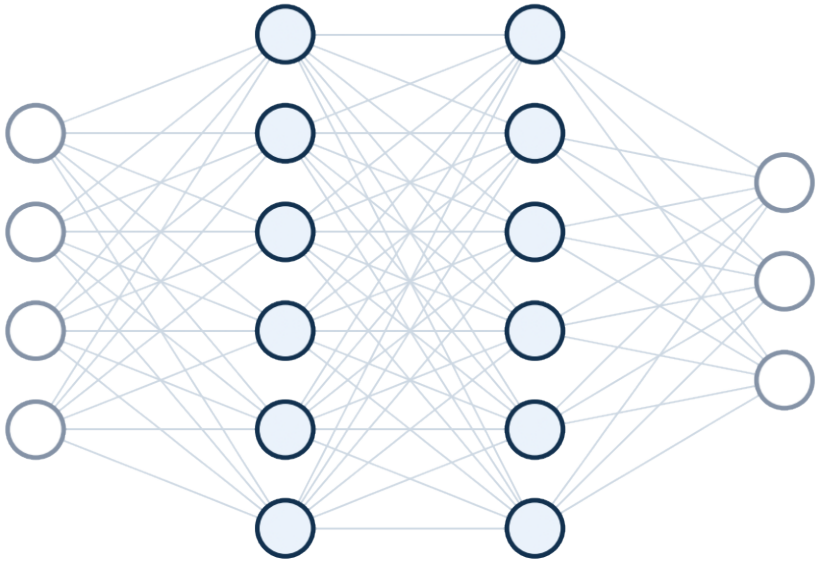
$$Z = XW + b$$



KEY IDEA Z = XW + b computes every neuron in the layer in a single matrix operation — this is exactly why GPUs make training fast.

Deep Network Architecture

Stacking dense layers builds depth; widening them builds capacity per layer.

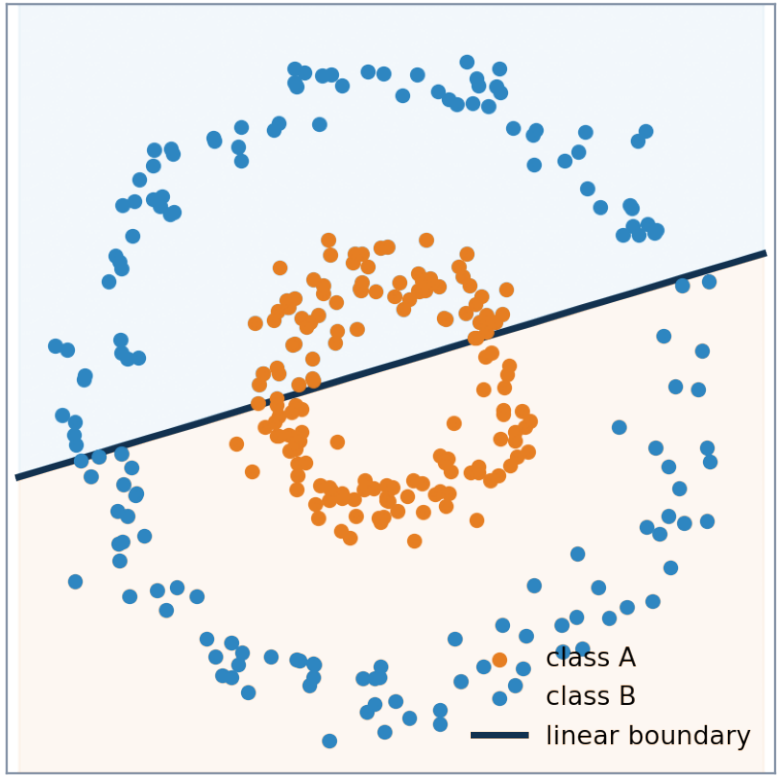


KEY IDEA depth lets the network compose simple functions into complex ones; each hidden layer transforms the representation passed to it by the layer before.

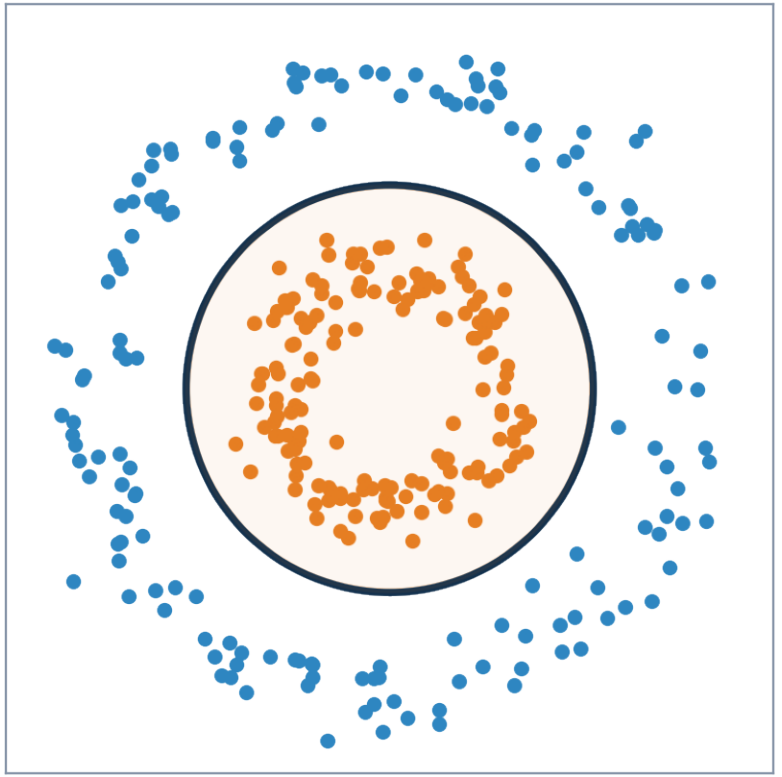
Why Activation Functions Matter

Without activations, stacking any number of linear layers still gives one straight line.

Without activations: only straight lines



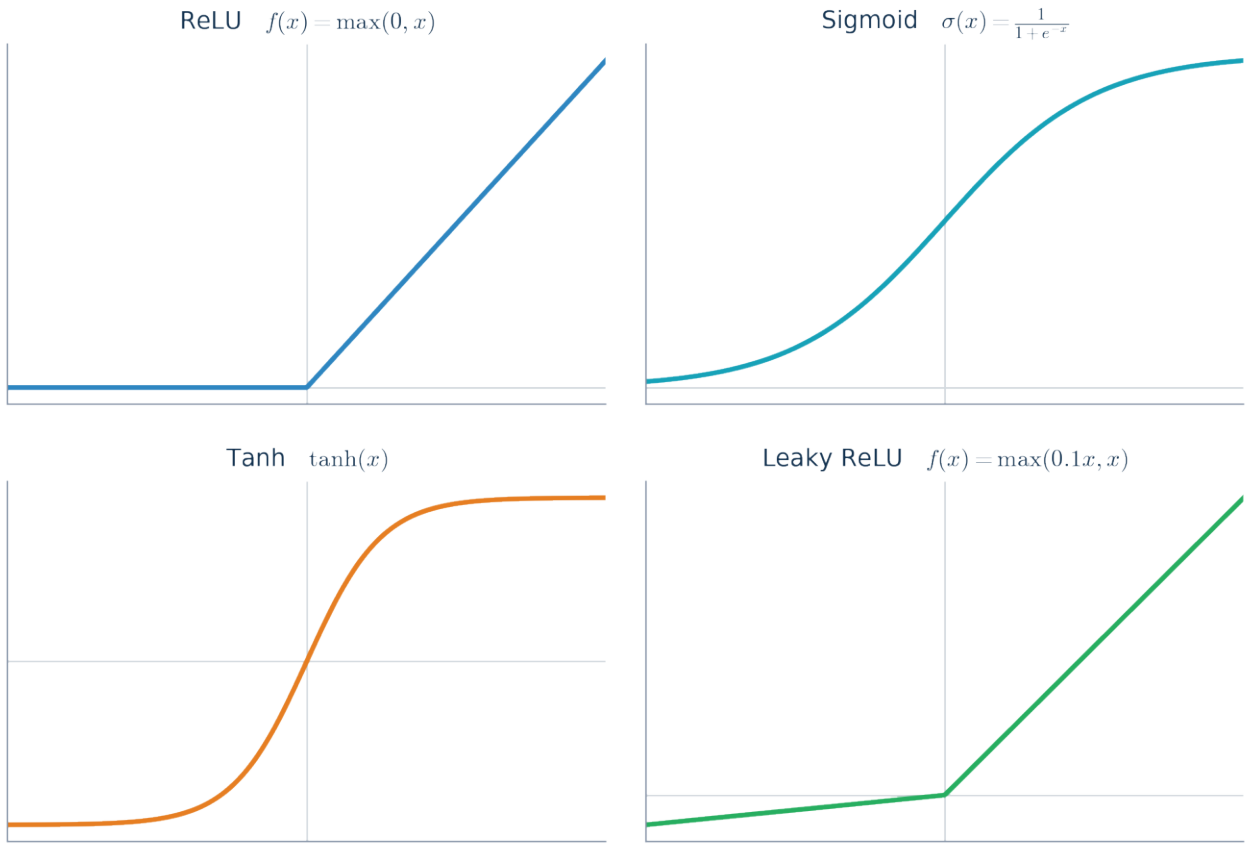
With activations: curved boundaries



KEY IDEA activations let the network bend and curve its decision boundary — this is what makes deep networks more powerful than plain linear regression.

Activation Functions

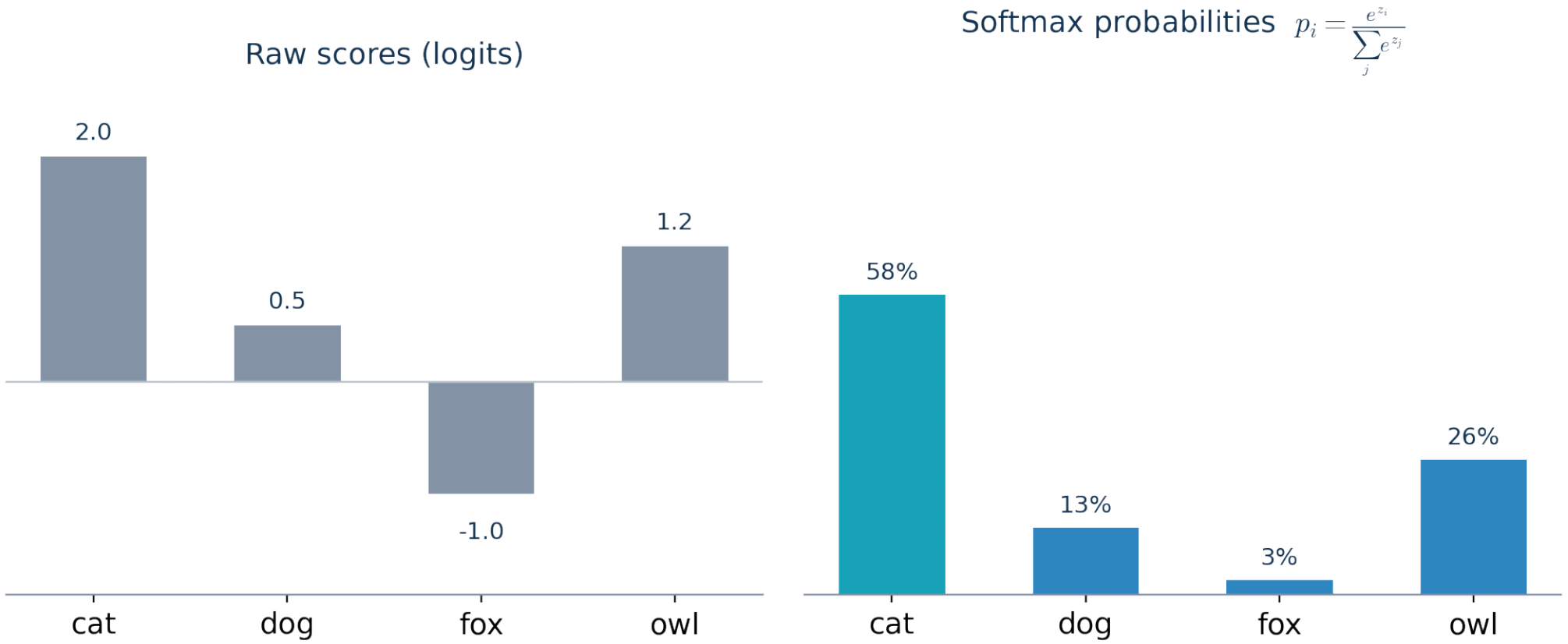
Different activations shape a neuron’s output in different ways.



KEY IDEA ReLU is the default for hidden layers (cheap, avoids vanishing gradients); sigmoid/tanh saturate at the extremes; softmax is reserved for the output layer.

Softmax — From Scores to Probabilities

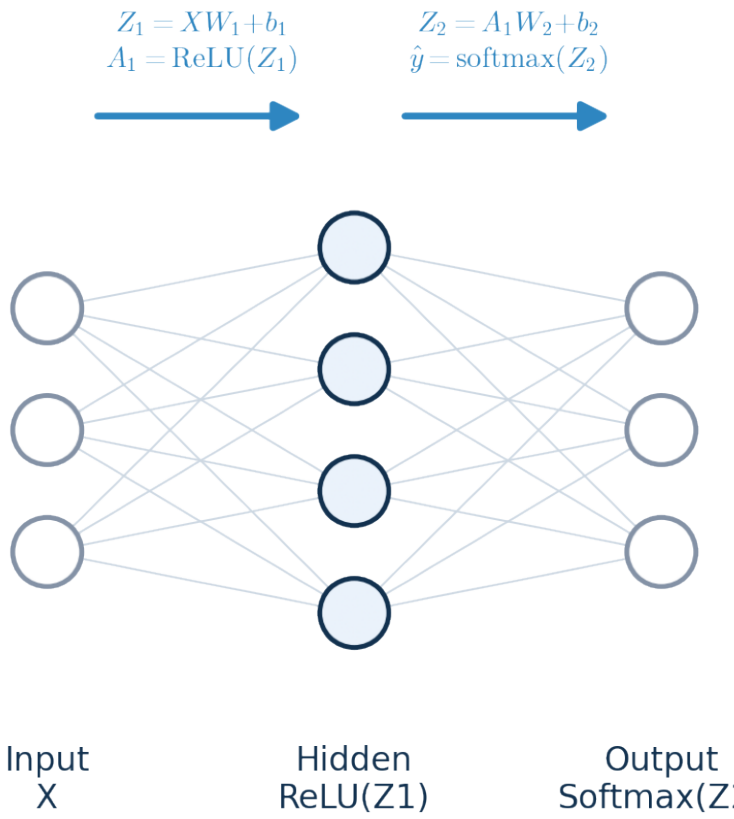
The output layer turns raw class scores into probabilities that sum to 1.



KEY IDEA softmax exaggerates the gap between the largest score and the rest, so the network expresses confidence, not just a ranking.

Data Flowing Forward

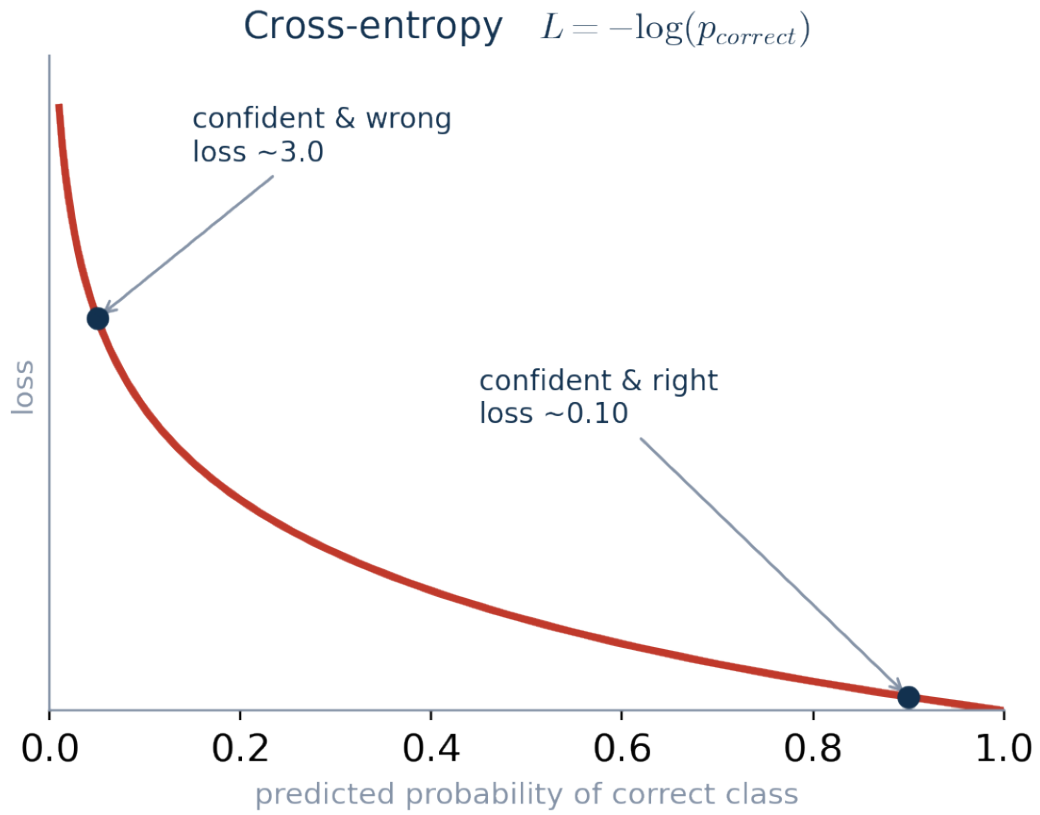
The forward pass pushes data through every layer, left to right, to a prediction.

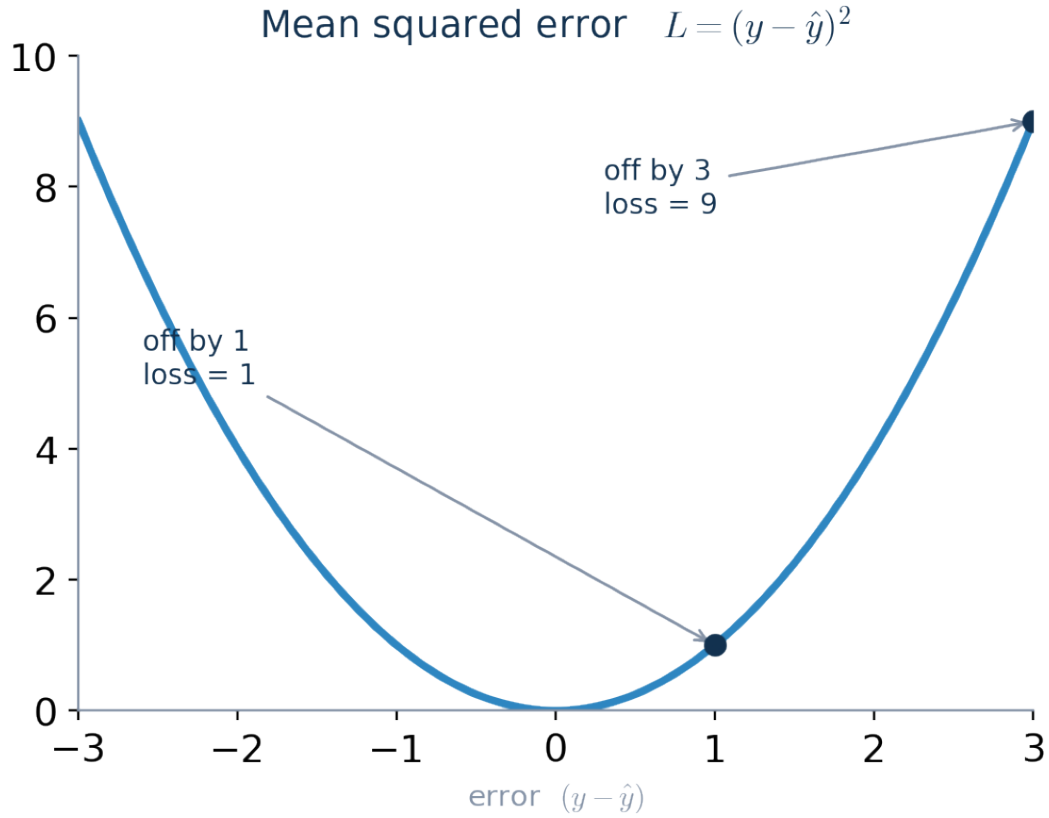


KEY IDEA each layer applies Z = AW+b then an activation; the final activation is what you choose based on the task (softmax for classification, none for regression).

Measuring Wrongness — Loss Functions

Loss converts a prediction's error into one number the network can shrink.

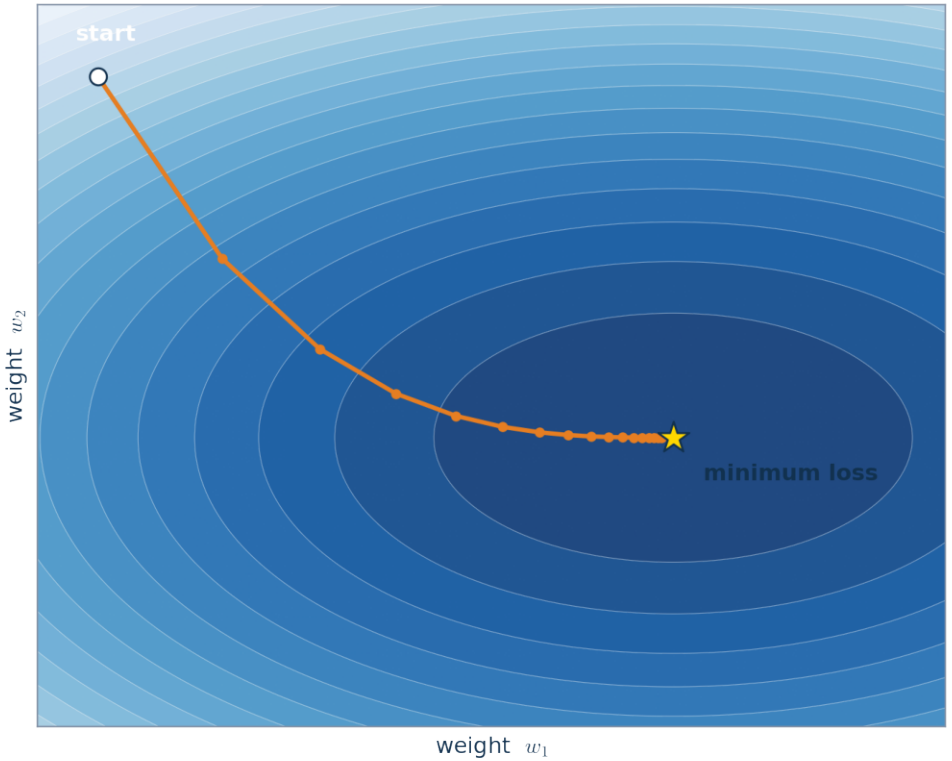




KEY IDEA cross-entropy punishes confident wrong answers severely; squared error punishes large numeric misses much more than small ones.

The Loss Landscape & Gradient Descent

Training walks downhill on the loss surface, one small step at a time.

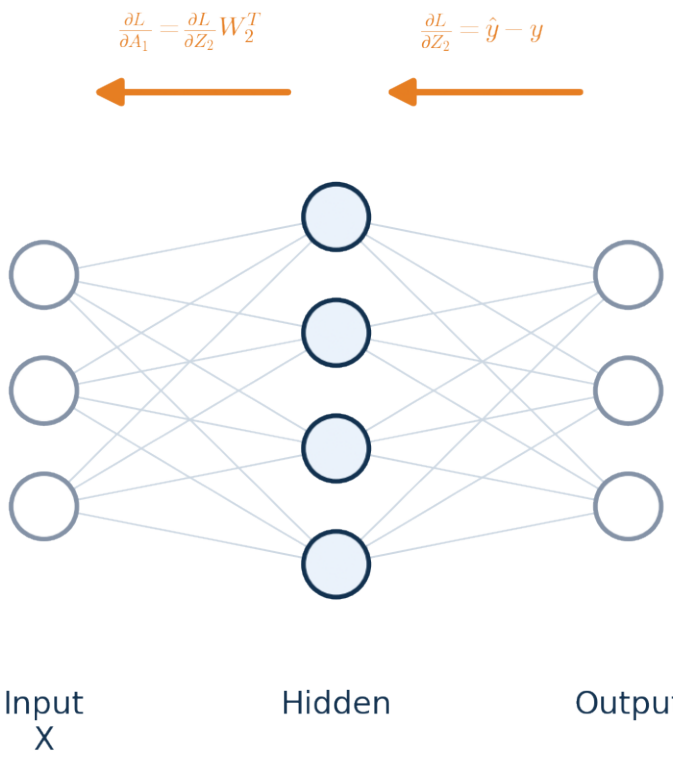


$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

KEY IDEA the gradient points toward increasing loss, so we step in the opposite direction: W <- W - eta * dL/dW, where eta is the learning rate.

Backpropagation — Passing Blame Backward

After the loss is computed, gradients flow backward through the same network.

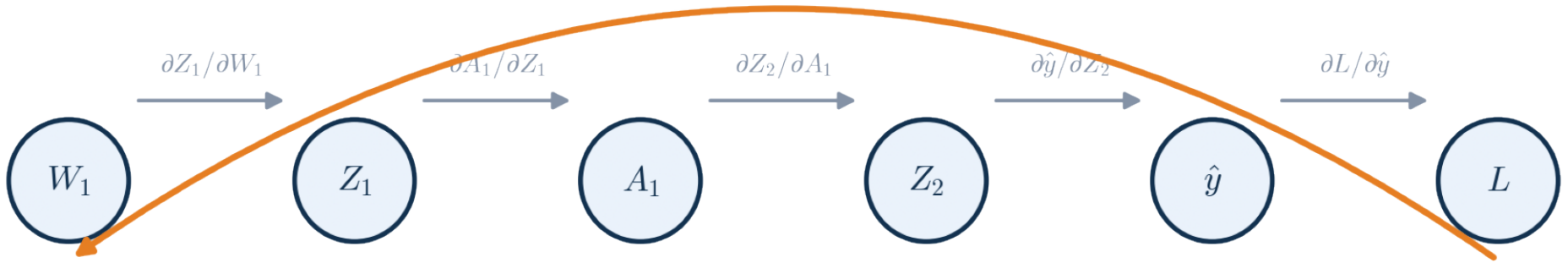


gradients flow backward, layer by layer, from the loss to every weight

KEY IDEA each layer receives the gradient of the loss with respect to its output, uses it to compute its own weight gradients, then passes a gradient further back.

The Chain Rule

Backpropagation is just the chain rule, applied link by link, back to every weight.

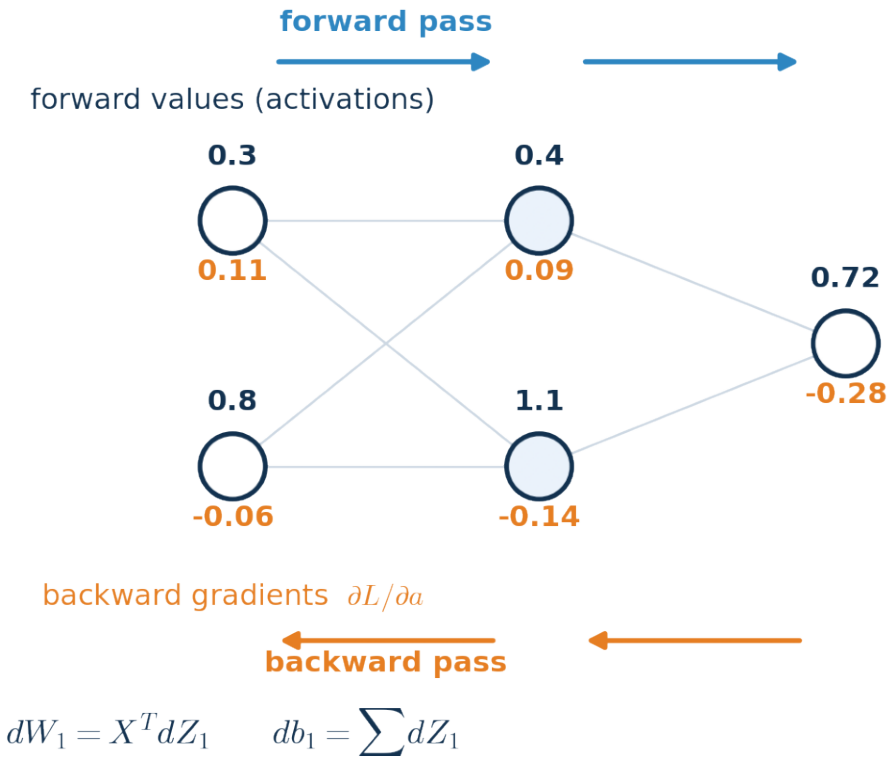


$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1}$$

KEY IDEA to find how W1 affects the loss, multiply the local derivative at every step between W1 and L — nothing more exotic than the chain rule from calculus.

Gradient Flow — A Worked Example

Forward values and backward gradients live on the same network, flowing opposite ways.

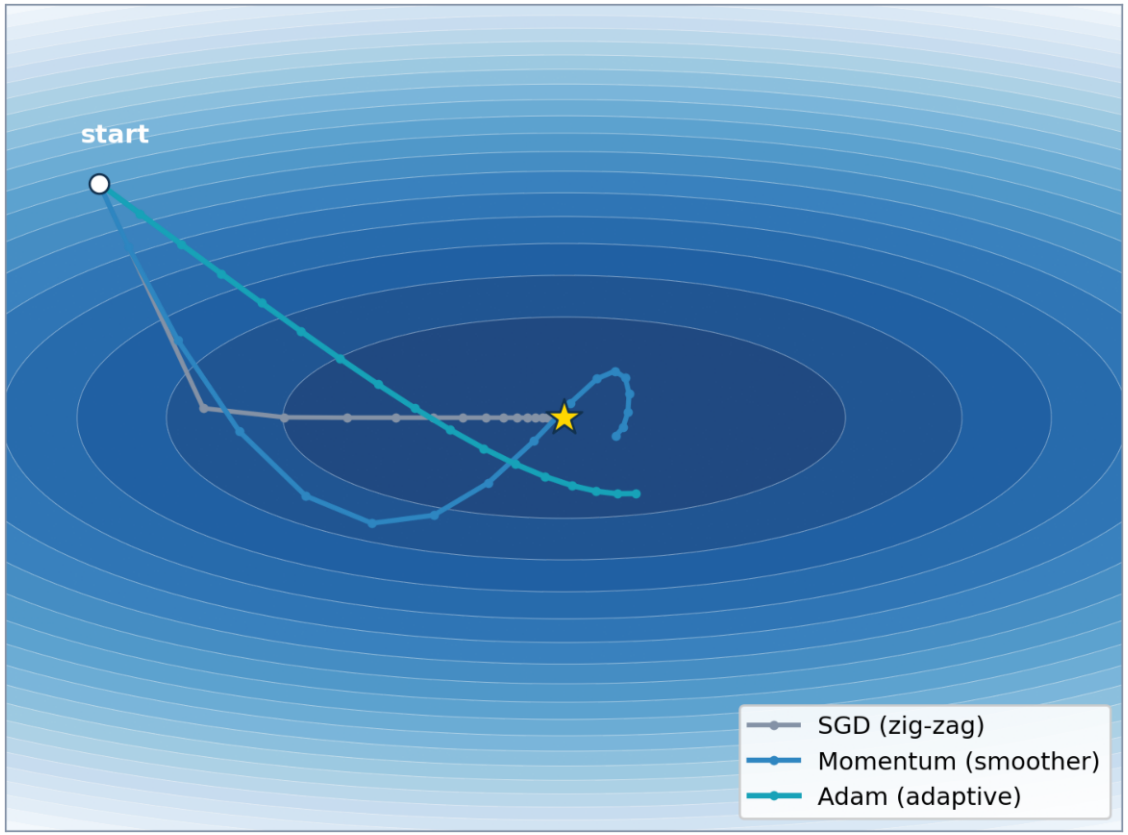


$$dW_1 = X^T dZ_1 \qquad db_1 = \sum dZ_1$$

KEY IDEA once every node has its local gradient dL/da, computing dW = X^T dZ and db = sum(dZ) for each layer is simple matrix arithmetic.

SGD vs Momentum vs Adam

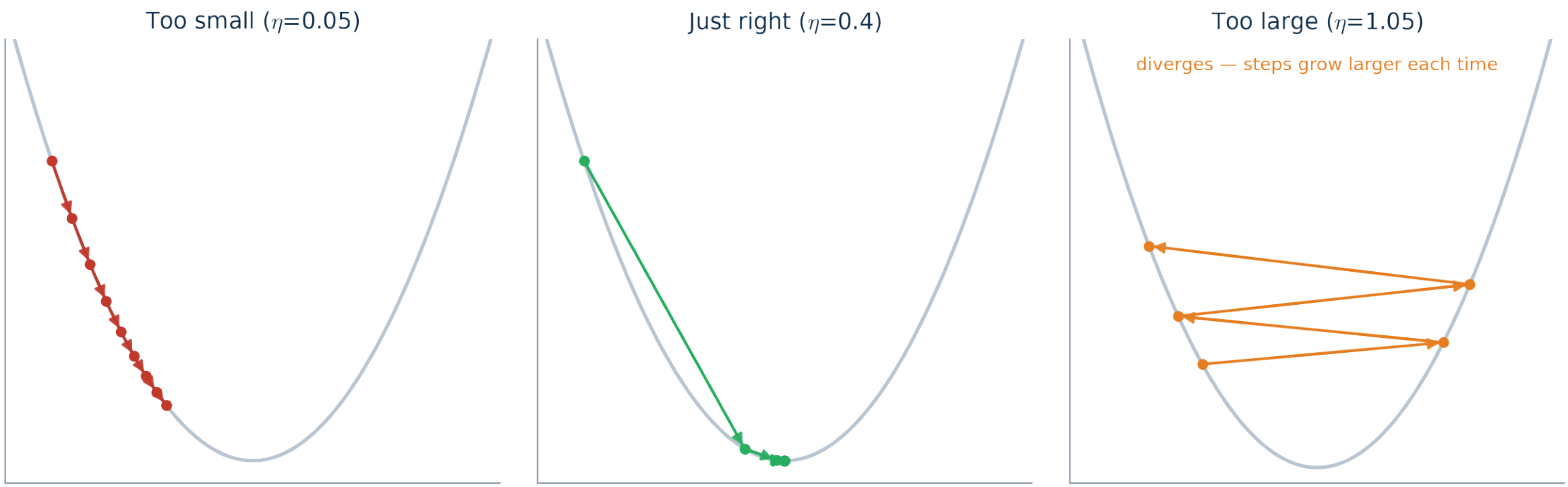
The same gradients can be turned into weight updates in very different styles.



KEY IDEA plain SGD can zig-zag across steep valleys; momentum smooths the path by remembering past steps; Adam adapts the step size per-parameter.

The Learning Rate

The learning rate eta controls how big a step the optimizer takes each update.



KEY IDEA too small crawls forever; too large overshoots and can diverge; the right learning rate is often the single most important hyperparameter to tune.

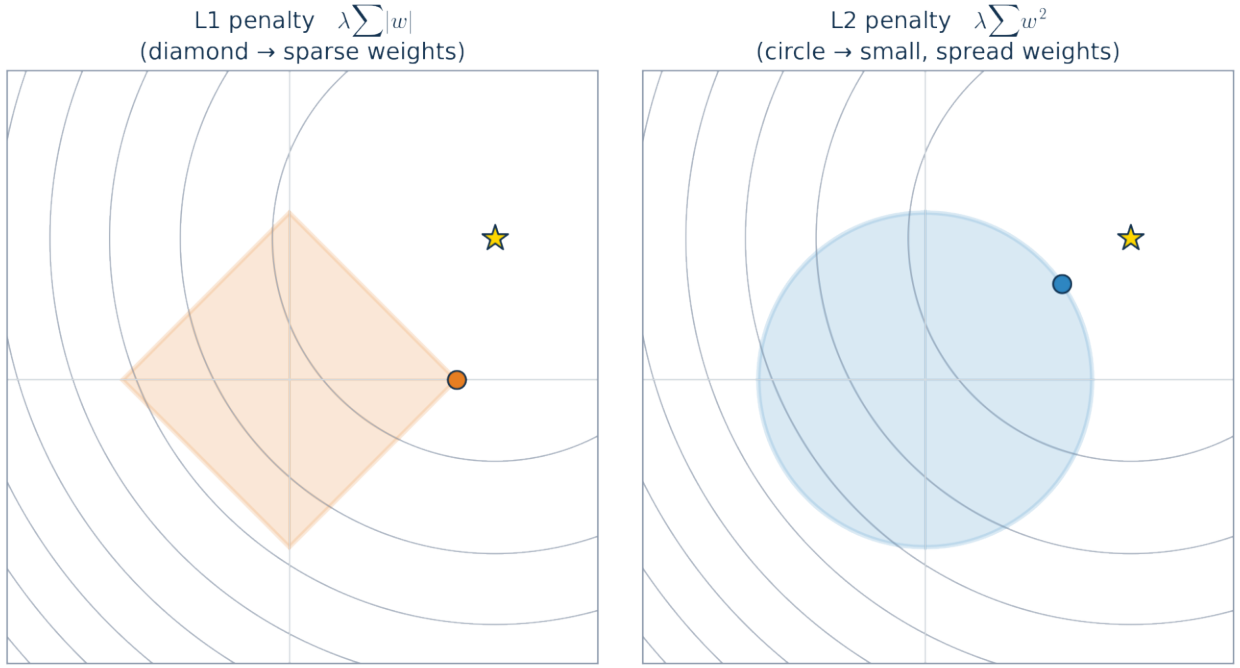
Overfitting vs Underfitting

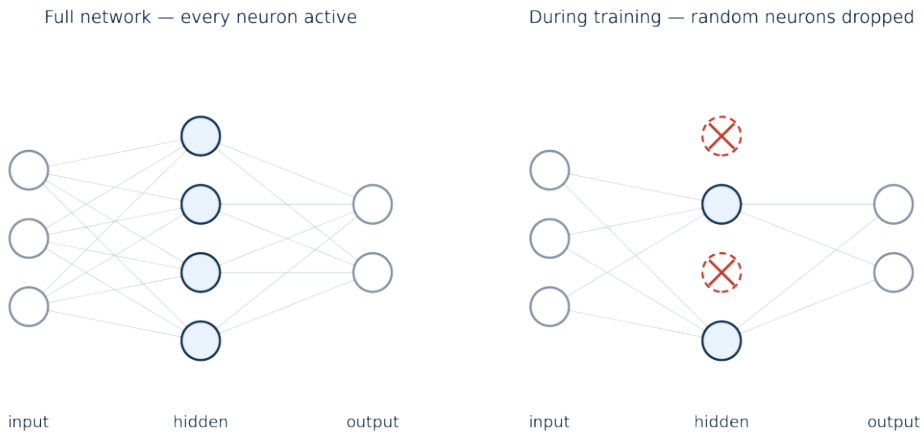
A model that memorizes its training data has not learned the underlying pattern.

KEY IDEA the goal is low error on NEW data, not zero error on training data — this is why we always evaluate on a held-out validation or test set.

Regularization & Dropout

Two tools that fight overfitting by discouraging the network from over-relying on any one signal.

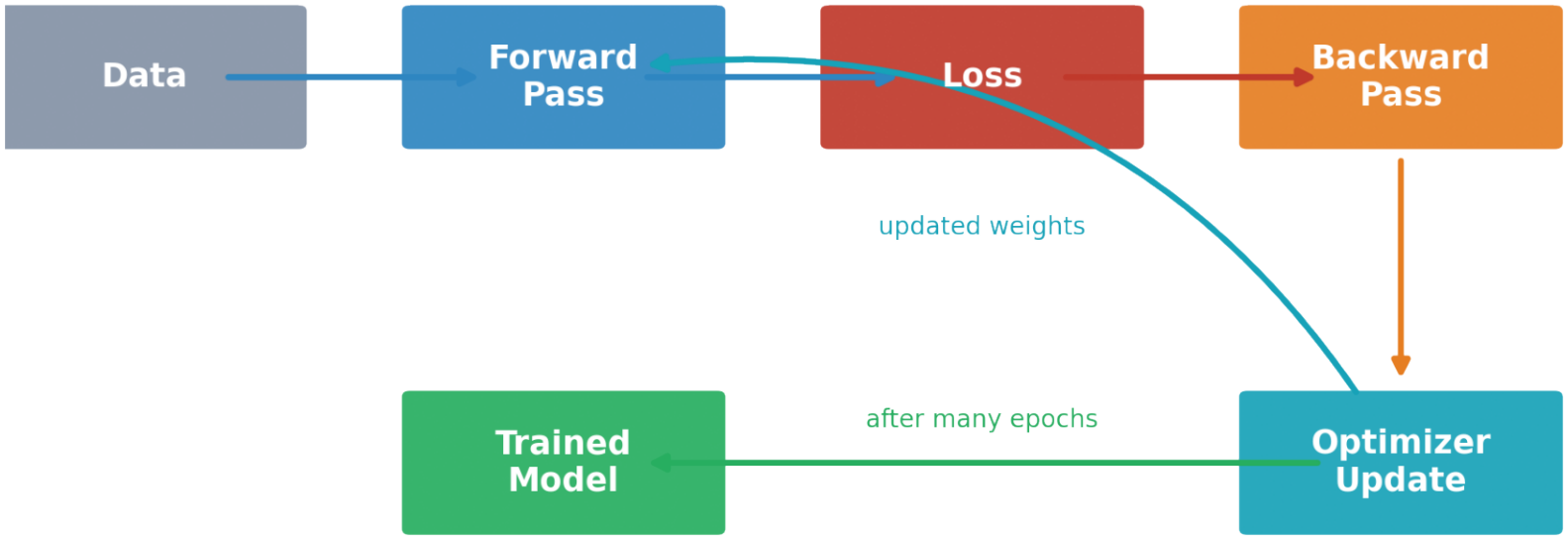




KEY IDEA L1 pushes weights to exactly zero (sparse); L2 keeps every weight small; dropout randomly disables neurons during training so no single path is indispensable.

The Complete Training Pipeline

Every concept in this deck fits into one repeating loop.



repeat thousands of times: forward -> loss -> backward -> update

KEY IDEA forward pass -> loss -> backward pass -> optimizer update, repeated over batch after batch, is the entire story of how a neural network learns.